

Coding with AI

Context, workflow, and **using AI in science**

Stephen Kerr

Computational Materials Science · Queen's University

The plan

- 1 Tokens and context** the window, and the handoff that curates it
- 2 From chat to agentic coding** the agent finds its own context
- 3 How I code with an agent** foundation · spec · slices · evidence
- 4 Skills as maintained assets** package the disciplines you repeat
- 5 Ownership and validation** the judgement that stays human



What is a token?

The smallest bit of information the model works with.

A node connected to the other tokens in the window.

Its working meaning comes from those connections.



How many r's in strawberry?

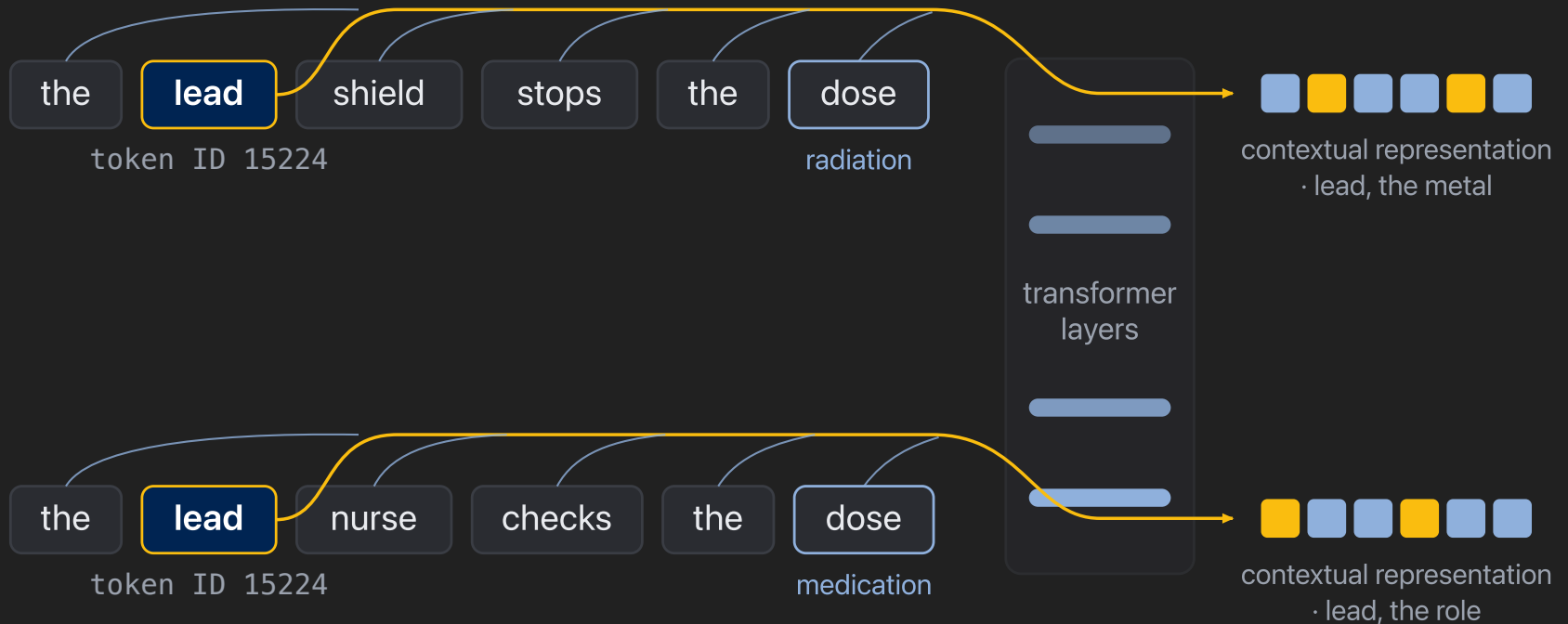
strawberry

A direct count asks the model to work below its own units: it sees chunks, not letters.

s t **r** a w b e **r** **r** y

When a model struggles: **change the representation**, expose intermediate steps, or give it a tool.

Same token ID, different meaning



Same ID in. **Different working meaning** out.

The token ID is fixed. The representation carried at its position becomes **contextual**.

Keep context small and relevant

context window



A million-token window is capacity, not a target.

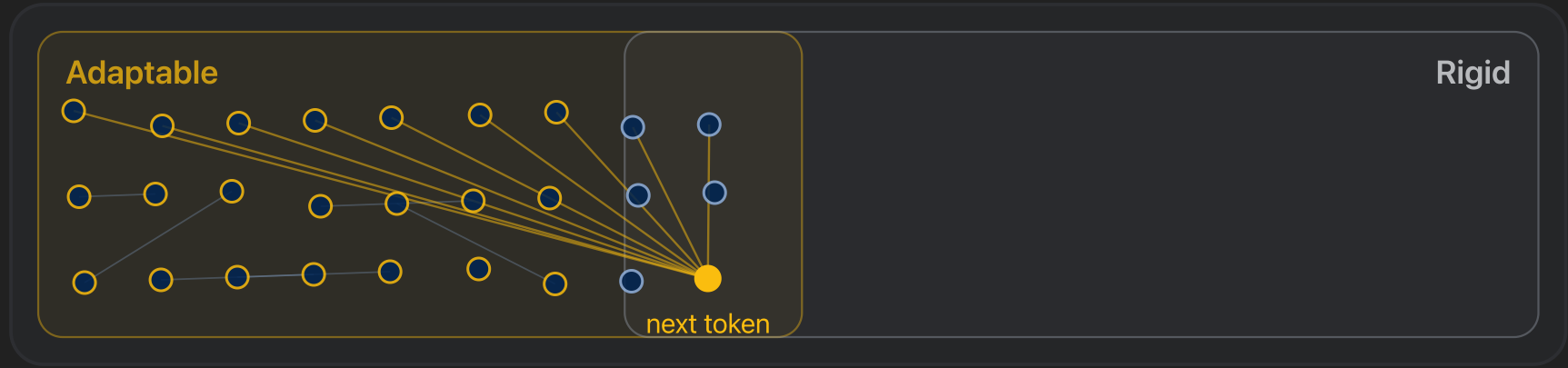
A fresh chat: small and targeted

context window



The chat grows

context window



Near capacity: attention spread thin

context window



A model's best work happens when the context is **small and targeted**.

Do not be afraid to start a new chat

New chat

A fresh, focused window.
Steering is easiest before the crowd moves in.

/btw side questions

Answered in an overlay: it sees the whole chat, but the answer never enters the window.

Carrying the thread into the new chat is a handoff.

A workspace the agent can navigate

Your workspace: a Project in the web app, or a codebase on disk.

One huge file

The whole textbook in a single document. No window can hold it.

Navigable chunks

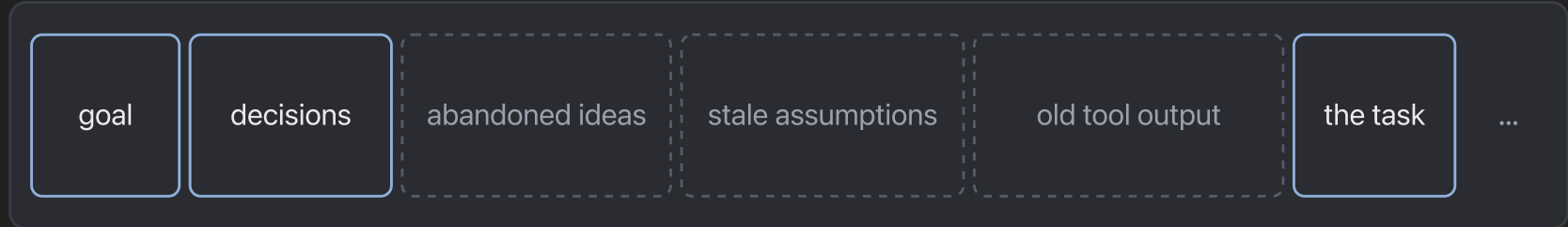
A file per chapter section. The agent parses the structure, then loads only what the task needs.

Structure the workspace so any one task fits a small window.

Handoff: compaction with intention

one long session

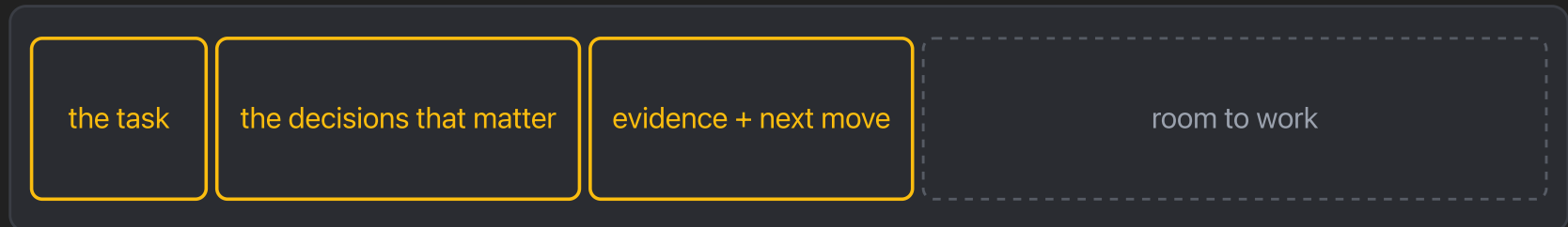
crowded and set in its ways



handoff: I choose what survives

a fresh session

focused and easy to steer



One bounded task, only the context needed to complete it.

Automatic compaction vs handoff

automatic compaction

fires when the window is
already full
a summary nobody curated
can silently drop a locked
decision
what was lost is invisible

handoff

happens when I choose
carries what the next task
needs
decisions survive because I
put them there
a file I can read and audit

Both compress the conversation. Only one is **intentional**.

One handoff, in practice: a diverged pyKMC bug

- A pyKMC bug was already fixed, but on a heavily refactored branch
- The fix could not be transplanted into `develop`: different seams

carried: the bug, narrowly stated

carried: produce a failing test first

carried: clues from the branch fix

carried: the constraint, a small native change

Carry the problem, evidence, and clues across the boundary,
not an obsolete solution.

What a handoff carries depends on where it is going

Direction

Might carry

Continue the same work

current state · settled decisions · useful evidence · next move

Break in a new direction

relevant premise · divergence point · local boundary · open question

Execute one piece of a larger plan

pointer to the plan · scope of the piece · starting state · verification target

Separate worktree

branch pointer · integration boundary · relevant checks

Examples, not required boxes. I choose what the next context needs.



Parallel work: handoffs + worktrees

same task: stale context → curated handoff → fresh chat, same code state

one planned piece: written plan → bounded handoff → fresh chat → verified result

parallel: independent handoffs → separate worktrees → merge + integration checks

A handoff scopes the **context**; a worktree isolates the **code state**. Neither makes integration automatic.

Vocabulary: tokens and context

Term	In this talk
token	The smallest bit of information the model works with. A node whose meaning comes from its connections.
token ID	The fixed integer produced by tokenization. It is not what changes.
contextual representation	The vector carried at a position after the layers mix in context. Two identical IDs can diverge.
context window	The model's finite working set. Select what should shape the next answer; do not fill it.
adaptable / rigid	My labels for session quality: focused and steerable, versus crowded and set in its ways.
/btw side question	Answered beside the chat: it sees the conversation, but never enters the window.
handoff	Compaction with intention: one bounded task, only the context needed to complete it.
automatic compaction	The safety net when the window fills: a summary nobody curated. Do not depend on it.
worktree	An isolated checkout. Separates code state, not conversational context.

“I need to wash my car, the carwash is 10 m away should I walk or drive?”

Walk or drive?

I need to wash my car, the carwash is 10 m away should I walk or drive?

At 10 metres, driving is irrational.

You would spend more time starting the car, shifting into gear, and positioning it than simply walking. There is also unnecessary engine wear and fuel use for essentially zero distance.

Walk. It is faster, simpler, and avoids pointless mechanical strain.

📄 ↺ ...

ChatGPT 5.3 Instant · 2026-07-13 · one observed response

The car has to arrive at the wash.

Walk... unless

I need to wash my car, the carwash is 10 m away should I walk or drive?

If the car wash is literally **10 meters away**, walking makes much more sense.

At that distance:

- Walking takes only a few seconds.
- Driving would likely take longer than the walk once you account for getting in, starting the car, and maneuvering it.

The one exception is if you mean you need to **bring the car to the car wash** because it's a drive-through or automated wash. In that case, you would, of course, drive the car the 10 meters into the wash.

So:

- If you're just going over there yourself: **walk**.
- If you're taking the car into the wash: **drive the car the 10 meters into the wash**.

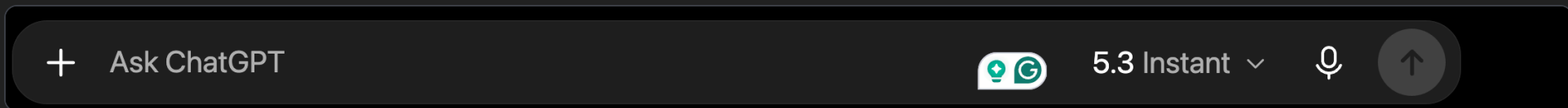
ChatGPT 5.5 Instant · 2026-07-13 · one observed response



Same prompt, different answers

The same 10 m question, other models and modes (July 2026):

Response	What it did
ChatGPT 5.3 Instant	Walk. The purpose never surfaces. (shown live)
ChatGPT 5.4 Pro	Drive: the car must reach the wash.
ChatGPT 5.5 Instant	Walk first, then the exception: the car has to be brought in. (shown above)
Claude Haiku 4.5	Walk.
Claude Sonnet 5 Low	Walk.
Claude Opus Low	Drive.



One correct answer to a familiar trap is **not evidence** the model will recover the hidden purpose of a novel request.

Inside the codebase

Web chat

Works from the context I give it. It sees the fragments I paste or upload, and nothing else.

Agentic coding

Works inside the codebase. It goes looking for the files and relationships the task needs, within the access it has been granted.

An LLM working in my codebase, sussing out where things are for itself. CLI or app: **the interface is not what makes it agentic.**

I was the agent

- Two courses, before any of these tools had agency
- One total energy, buried in a huge Quantum ESPRESSO output
- A grad-course analysis whose output file swamped the chat
- Both worked only once I supplied the procedure:

`grep the lines that matter → write them to a small file → analyse that → report back`

I was doing a subagent's job by hand.

Vocabulary: agentic coding

Term	In this talk
agentic coding	An LLM working inside the codebase, having sub-workers find relevant context for itself.
situated access	The agent inspects the repository, bounded by the access it was granted.
inspect · act · observe · adapt	How a coding agent behaves once it can see the work. A description, not the boundary.
web chat	Works only from what you paste in. Capable, but context-blind.

FOUNDATION.md: a context start

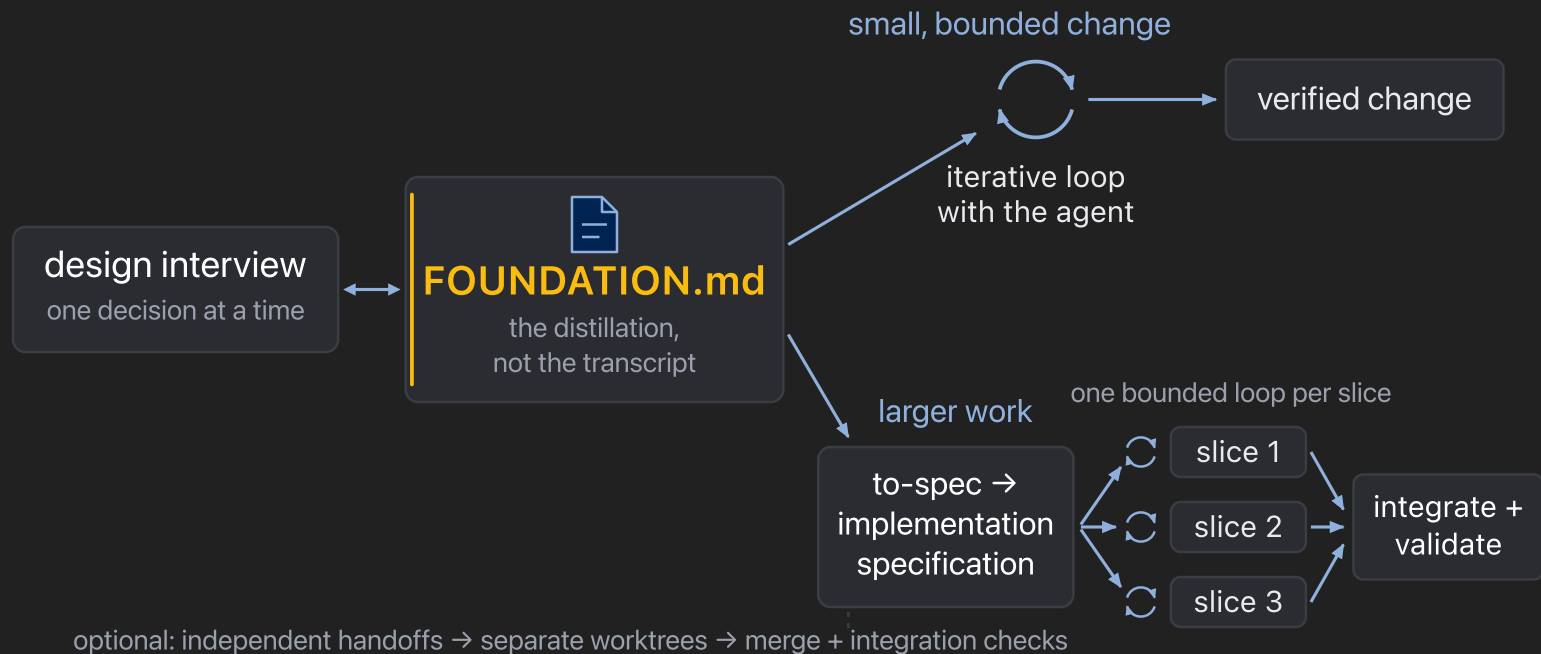
- The distillation, not the transcript
- Goal · glossary · locked decisions and why · boundaries · evidence · definition of done
- A new session loads the agreed design directly

Do not depend on automatic compaction for durable design.

Commit the north star to a file.

This presentation was built exactly this way.

Scale the process to the work



Every step hands the next agent a **smaller, sharper context**.

Ralph: the blunt end of the loop

```
while ! ./slices-done.sh; do
  claude -p "Read SPEC.md and PROGRESS.md.
    Take the next unfinished slice.
    Implement it, run its checks,
    update PROGRESS.md."
done
```

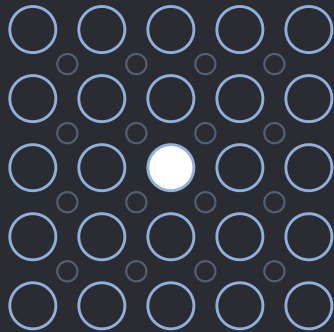
- Every lap starts with an empty context: the files are the memory
- Only works because the artifacts exist: durable spec, bounded slices, observable checks

Autonomy is earned by the artifacts, not by the loop.

One slice, one owned decision

coordination_threshold = 8

FCC bulk

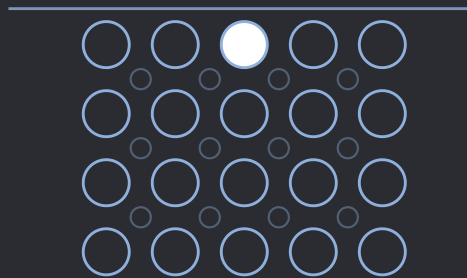


12

nearest neighbours

ordinary FCC(100) surface

vacuum

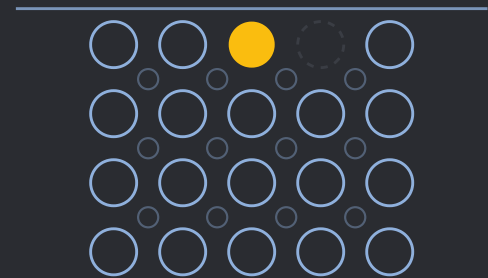


8

still crystal at threshold 8

beside a surface vacancy

vacuum



7

below threshold →
search events here

A specification does not make the scientific decision. It **preserves** it, and turns it into explicit, testable behaviour.

Do not fix what you cannot reproduce

- First: one command that fails on the exact symptom
- Minimise it; keep it as a regression test at the right seam
- Only then: the smallest fix, and the same evidence turns green
- Re-run the original scenario

The agent must prove it sees the same bug I see.

Vocabulary: the workflow

Term	In this talk
design foundation	Durable Markdown distillation of the agreed design. A context start for future sessions.
implementation spec	The agreed end state packaged for execution. Study or method spec for simulation campaigns.
vertical slice	A narrow, complete capability, independently verifiable. A file or layer is not a slice.
red-capable reproducer	One runnable command that fails on the exact reported symptom, then passes after the fix.

A prompt earns its packaging

- A discipline I repeat, preserved as a Skill
- Exposed only to the chats where it is relevant
- Observed for drift; revised or removed when it stops helping
- Not only coding: one Skill adversarially reviews my statistical claims before I draft prose

Not something to collect. **Something to maintain.**

Expose the decisions

Skill: interview-design-decisions

- The discipline: no code until the decisions are on the record
- The agent interviews me: one consequential choice at a time
- It proposes options and recommends one
- I lock in the load-bearing decisions

Surface the decisions the agent would otherwise guess.



A recorded decision is a findable mistake

HTST rates for pyKMC: the interview forced the consequential choices into the open.

- One question settled the time scale: I took the paper's convention, and the spec recorded it
- Later, a computed rate landed orders of magnitude off an accepted value
- Asking *why* did not need archaeology: the choice that went wrong was on the record

The Skill did not catch the bug. **It made the wrong decision findable.**

The Skill, as written

No magic in the file: prose that tells the agent how to treat me.

```
Interrogate me about each decision that is not  
already settled in the code base or foundation.  
One question at a time. Offer real options and  
say what each one changes. Wait for my decision,  
then record it. Stop when I can defend every choice.
```

It hunts the decisions **not yet on the record**.



What does this intern need today?

Study chat

Textbook read-aloud Skill: preserve the spoken text, skip figures, expand acronyms.

Hobby chat

A hobby's own Skill shelf, just as carefully maintained. Nothing a repository needs.

Coding chat

Debugging Skill: reproduce the exact failure before repairing it. What this repository needs.

Not good Skill versus bad Skill: **context this intern should see today**, versus context it should not.

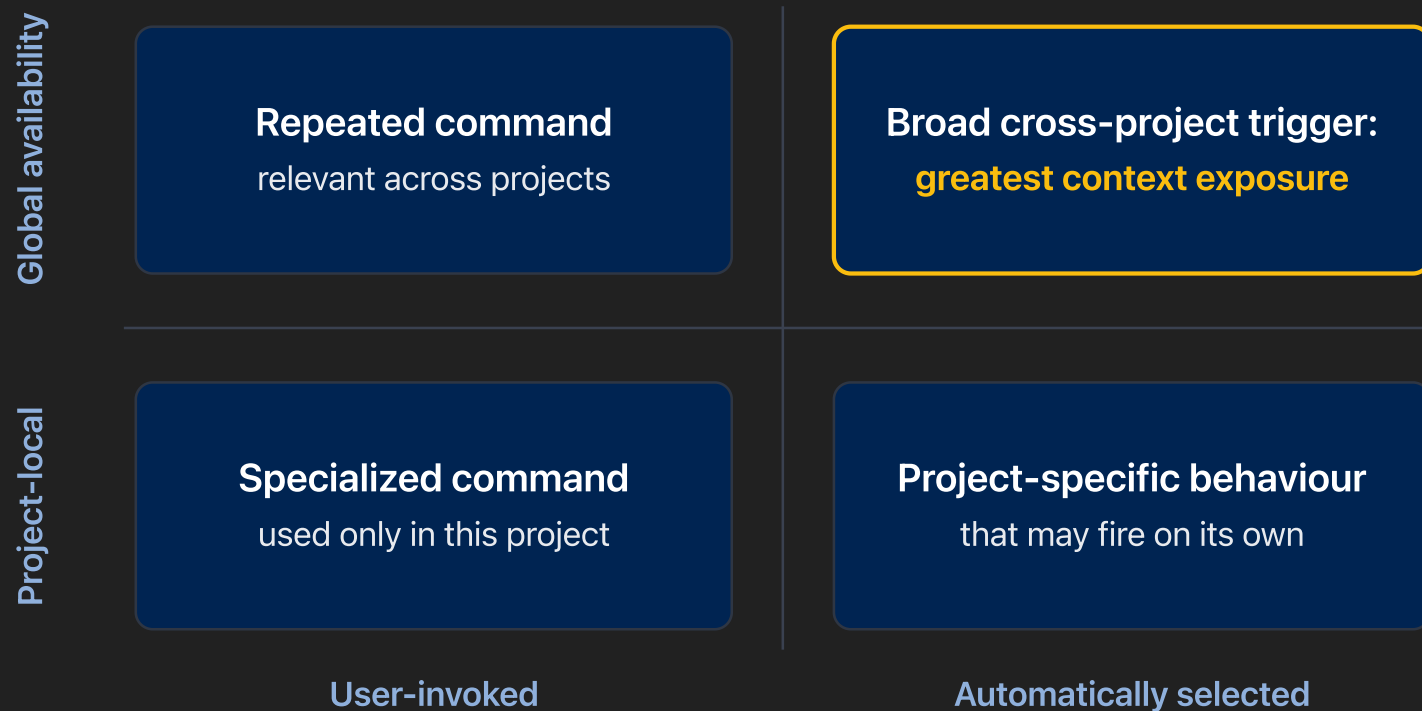
Installing a Skill expands what the agent may **trust, trigger, and run**

Agents are eager: give one a tool and it will look for a reason to use it.

- Listed in a CLAUDE.md or AGENTS.md file? Then every agent, and every subagent it spawns, has it on the desk
- Instructions enter the trust boundary: possibly scripts, hooks, network access

Inspect before you adopt. Keep the active set small.

Two axes, one trigger surface



Trust is a prerequisite at **any** scope. After that, placement is contextual relevance.

Still the wild west

- No settled practice: Skill sets, project instructions, file conventions all differ
- Different arrangements really do produce different results
- How we work now is not how we will work in a few years

My configuration is a starting point, not a standard. Keep the habit: decide what belongs in context, and change it as the tools change.

One shelf, under version control

- Every Skill I maintain lives in one repo; each project checks out only its shelf
- It helps generate new Skills tailored to a given project
- It tracks which Skills are related, so an improvement to one is disseminated to the rest



github.com/Kerrator/skills

Open to the group: a jumping-off point for **a workflow of your own.**

Vocabulary: Skills

Term	In this talk
Skill	A discipline I repeat, packaged so the next chat does not rediscover it. Maintained, not collected.
trigger surface	Everything a chat can see that can cause behaviour. Every place a Skill is listed makes it larger.
invocation mode	Runs when I ask, versus selected by the model from its description.
installation scope	On one project's desk, or on every desk I own. A different axis from invocation.

Less connected to their own writing

MECH 479, across course offerings:

AI as orientation → AI as critic → AI as end-to-end author → **outer space**

- Rubric-shaped reports, produced very quickly
- Weak answers to questions about their own cited sources

The assignment stopped guaranteeing that a polished deliverable reflected understanding. **Redesign the checkpoints, not the blame.**

Stay inside the epistemic loop

The point where I have delegated so many intermediate judgements that I can no longer audit, explain, or defend the result.

I call it outer space.

Better structure, wrong comparison

AI restructured my dipole paragraph. Genuinely better writing.

A > B **became** A < B

the shape of the error, not the sentence

I kept the structure and repaired the science. Possible only because **I still knew what the sentence was supposed to say.**

Green tests can agree on the wrong physics

$$5 \times 10^{12} \text{ Hz} = 5 \text{ ps}^{-1}$$

- The Vineyard prefactor came out in Hz; the rate layer consumed the number as ps^{-1}
- Every finite HTST rate 10^{12} too large; simulated time 10^{12} too short

The tests did not stop me. A value I already had reason to expect did.

The unit boundary, in full

$$\nu_0 = \frac{1}{2\pi\hbar} \frac{\prod_i \hbar\omega_i^{\min}}{\prod_j ' \hbar\omega_j^{\text{saddle}}}$$

- The primed saddle product omits the imaginary mode: one energy dimension survives, so ν_0 carries units
- The error multiplies every finite HTST and RPA rate; constant-style runs and the k_0 fallback were unaffected
- The fix converts $\text{Hz} \rightarrow \text{ps}^{-1}$ exactly once, at the backend boundary, pinned by a Ni(100) regression against a known 5 THz prefactor

Public evidence trail

github.com/hugomoison/pyKMC · verified 2026-07-13

Reference What it shows

d705932 Hz $\rightarrow$ ps⁻¹ converted exactly once, at the backend boundary

d6b921f Regression: rate, residence time, constant-versus-HTST equivalence at the same prefactor

PR 71 The historical HTST design record (a closed, unmerged draft)

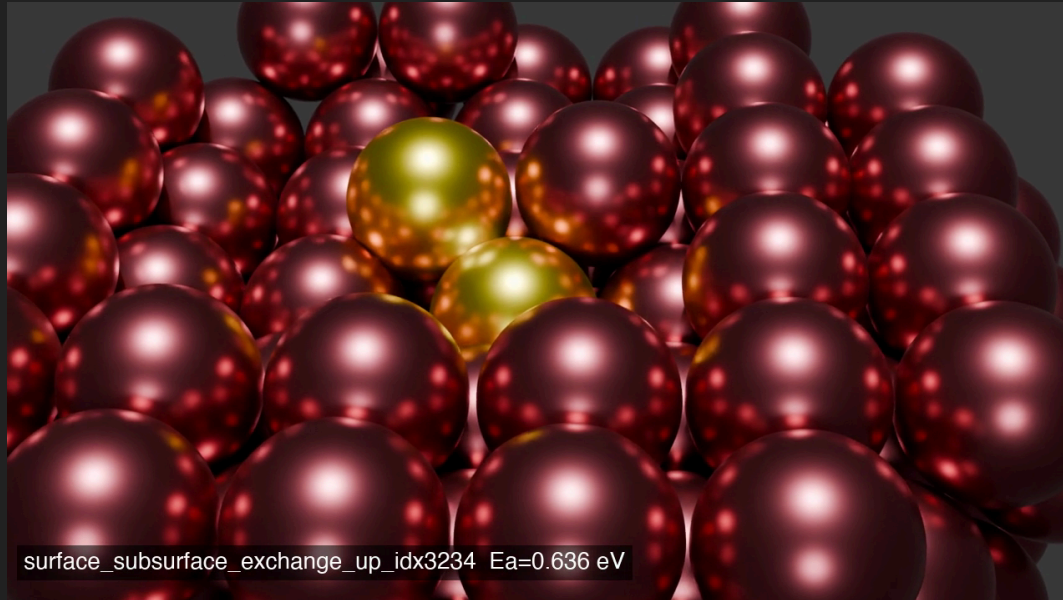
The links verify the implementations. **The disposable handoffs between them are my account.**



Vocabulary: ownership

Term	In this talk
epistemic loop	The chain of judgements you can still audit, explain, and defend.
outer space	My name for the far side: so much delegated that the result can no longer be defended.
external anchor	A value you already had reason to expect. It can catch what green tests encode wrongly.
regression vs validation	A pinned, machine-checkable expectation, versus an end-to-end confirmation run. Different strengths of evidence.

Going from idea to implementation has never been easier



blender



to-do



Netrunner



lotusfield

Delegate labour, not scientific judgement.

- Explain the choices
- Trace the evidence
- Challenge the output
- Say what observation would prove it wrong

Thank you. Questions?